Conceptual Architecture of WezTerm

Six subsystems · two use cases · one architectural story

WEZTERM WARRIORS

Presented by Miles Chant and Eric Charest

Presentation link: https://youtu.be/cEuywLlwL_Q

Our Group Roles

Benjamin Van Mil	Group Leader
Eric Charest	Presenter
Miles Chant	Presenter
Abraham Li Wu	
Owen Macey	

What WezTerm needs to do

Purpose and quality drivers

A terminal emulator with additional responsibilities

1	**GPU-accelerated rendering**	Fast, high-quality text
2	**Cross-platform support**	One codebase, many operating systems
3	**Lua extensibility**	User-defined logic and bindings
4	**Built-in multiplexing**	Tabs, panes and remote sessions
5	**Low latency**	Responsive even under heavy output

How we derived the architecture

Recovery method

1

Read primary sources

Documentation + repository structure

→

2

Identify responsibilities

Objectives → six conceptual subsystems

→

3

Map dependencies

Rust crates → box-and-arrow diagram

→

4

Validate with behaviour

Trace two use cases and control handoffs

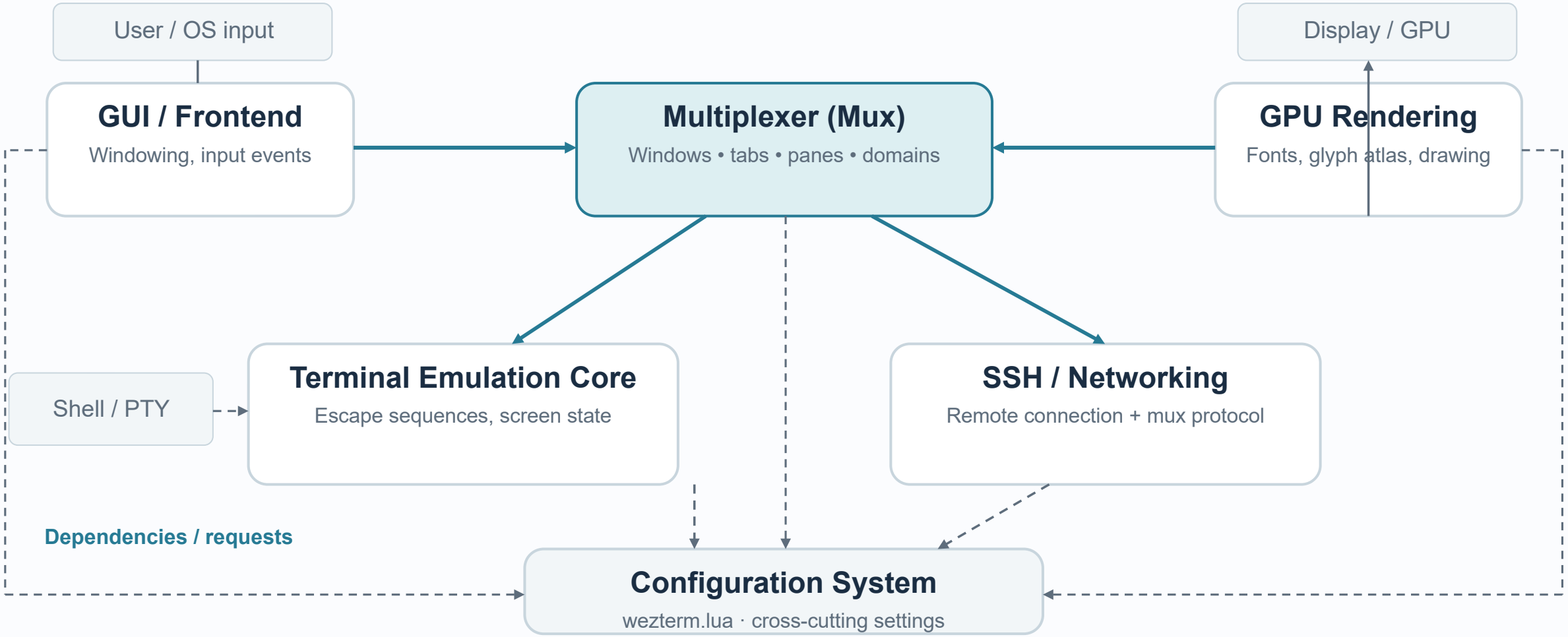
Scope of A1

Conceptual subsystem roles and interactions — not a complete code-level dependency analysis (reserved for A2).

System at a glance

Top-level box-and-arrow diagram

Six subsystems + the external environment



Solid = uses / requests · Dashed = supporting link to configuration / PTY

Why these architectural styles?

Layered, component, event-driven, pipeline

Layered

GUI → Mux → terminal logic

Limits what the GUI needs to know

Component-based

Subsystems map to Rust crates

Separate concerns and testing

Event-driven

Mux publishes change notifications

GUI does not poll each pane

Pipeline

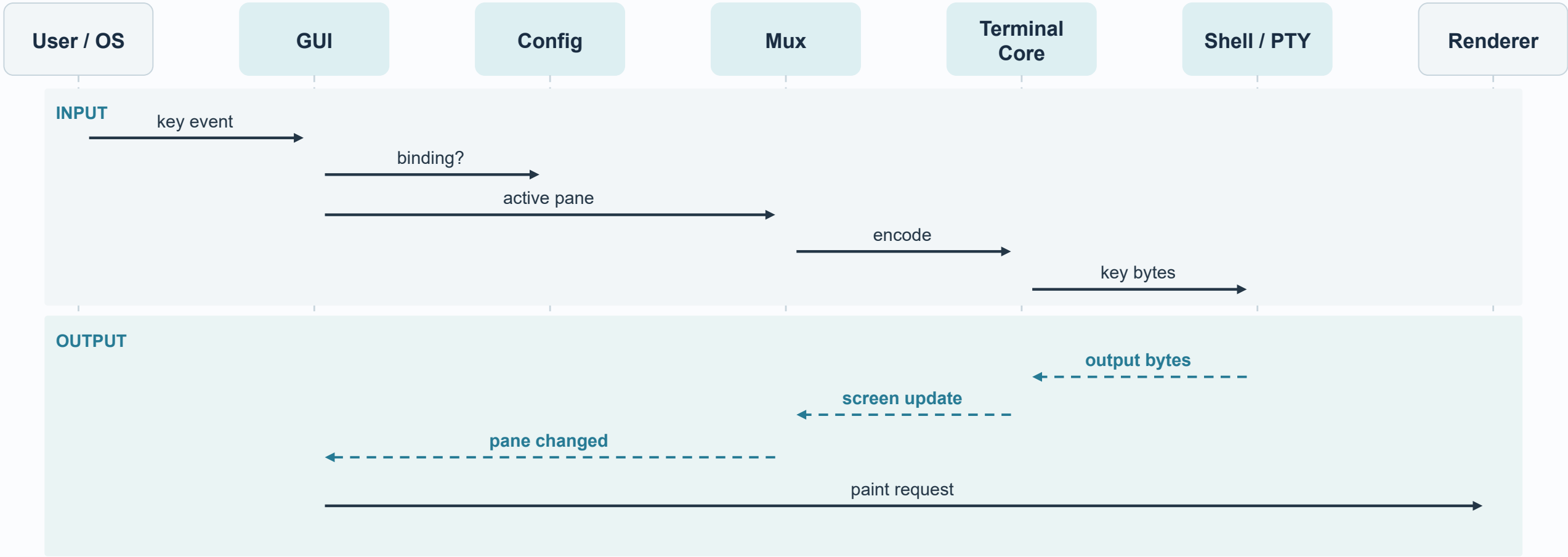
Bytes → screen → glyphs → frame

Stages can be handled independently

Key rationale: the Mux mediates sessions; configuration crosses subsystem boundaries.

Sequence 1 — Typing a key

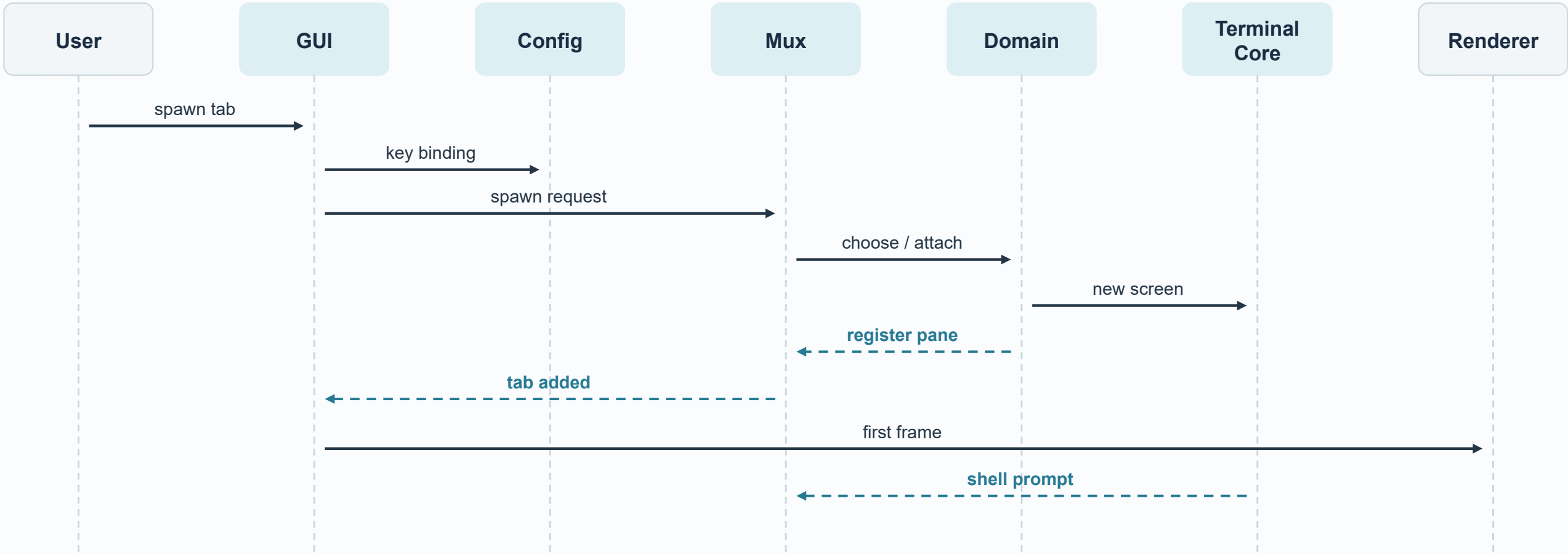
Input and output are decoupled



Solid arrows = request / data · Dashed arrows = asynchronous return / notification

Sequence 2 — Opening a new tab

Creation through a domain

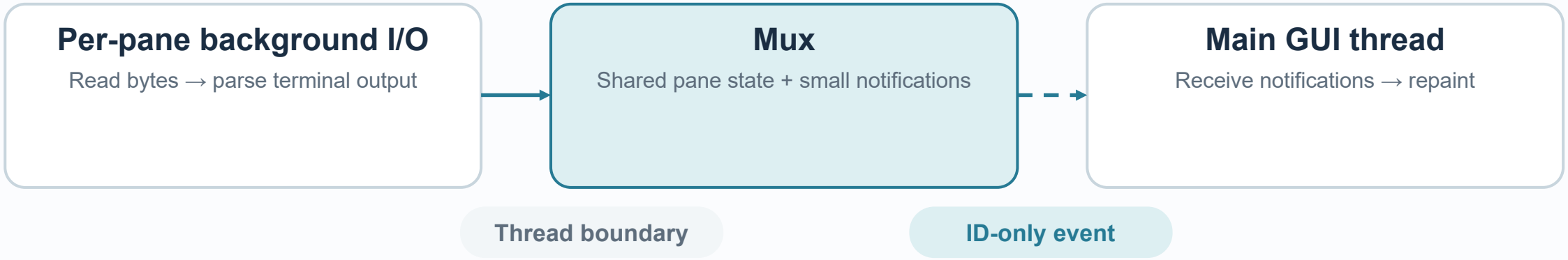


Domain = local PTY or remote connection; subsequent prompt follows sequence 1.

Concurrency & data movement

Responsiveness through decoupling

The GUI stays responsive while panes produce output



Benefit

Heavy shell output need not block keyboard input or drawing.

Coordination cost

Synchronized registries; avoid blocking Lua event handlers.

Evolution & developer ownership

Modularity is useful, not absolute

Lua configuration

Hot reloading + event handlers

Users adapt behaviour without editing Rust

Platform abstractions

Common windowing interfaces

Operating-system changes stay localized

Mux domain interface

Local and remote session types

New domain support behind a shared contract

Developer responsibility: maintain shared interfaces and test subsystem interactions.

Alternatives, trade-offs & limits

What this design gains and costs

Alternatives assessed in our report — not claims about historical design decisions

Built-in mux vs. External tmux

Native panes + integration | larger codebase

Cross-platform abstractions vs. Separate native apps

Shared code | platform-specific complexity

Lua as configuration vs. Fixed settings file

Extensibility | compatibility obligations

Limits: GPU drivers, a mostly single-maintainer codebase, and conceptual claims awaiting A2 validation.

Lessons, AI collaboration & conclusion

What we learned

Lessons learned

1. Agree on shared subsystem names and crate mapping early.
2. Use code-level diagrams to learn, not as the A1 conceptual picture.
3. Set page budgets before merging everyone's sections.

AI teammate: used, then verified

Claude Opus 5.5 · Sonnet 5.5 · GPT-6

Research guidance, drafts, diagrams, and integration

~40% of the production process aided by AI

Primary sources + human review remained essential.

Conclusion: responsiveness, portability and extensibility — with explicit complexity and maintenance costs.